Information Retrieval Lab Manual: 9 Practicals

Nov 4, 2026 · @rohit

Setup (do this once)

All nine programs run on Python 3.10 or newer, and each one is a single file you can run on its own.

1. Install Python from python.org and tick "Add Python to PATH".
2. Open a terminal and install the libraries: `pip install requests beautifulsoup4 pillow numpy matplotlib`
3. Make one folder, for example `IR_Lab`, and save each program in it as `p1_conflation.py`, `p2_single_pass.py`, and so on.
4. Inside `IR_Lab`, make a sub-folder named `documents` and put 4 or 5 small `.txt` files in it. Practicals 2 and 3 read from this folder.
5. Run a program from the terminal inside `IR_Lab`: `python p1_conflation.py`

Sample files used for the outputs shown in this manual:

File	Content
d1.txt	Cricket match won by India with great batting and bowling in the stadium
d2.txt	The cricket team practiced batting and bowling before the match
d3.txt	Python programming language is used for machine learning and data science
d4.txt	Machine learning models in Python need data for training
d5.txt	Football match ended with a late goal in the stadium

Practical 1: Conflation algorithm

Aim: generate a document representative (its most important terms) for a text file.

Idea: conflation merges word variants into one form, so "search", "searched" and "searching" count as the same term. The most frequent merged terms then represent the document.

Steps

1. Read the text file and convert it to lowercase.
2. Split it into words (tokenize).
3. Remove stop words such as "the", "is" and "of".
4. Stem each word by cutting common endings (-ing, -ed, -es, -s) so variants share one stem.
5. Group the variants under each stem, which gives the conflation classes.
6. Count how often each stem occurs.
7. Take the top N stems as the document representative.

Code (`p1_conflation.py` , creates `sample.txt` on first run; replace it with your own text file)

```
import re
from collections import Counter, defaultdict
from pathlib import Path

STOP_WORDS = {"a", "an", "the", "is", "are", "was", "were", "be", "been",
"of", "and",
              "or", "to", "in", "on", "for", "with", "as", "by", "at", "it",
"its",
              "this", "that", "these", "those", "from", "into", "can", "has",
"have",
              "had", "not", "but", "also", "than", "then", "so", "such",
"their",
              "they", "which", "who", "will", "would", "we", "you", "i"}

# longest suffixes first so "ings" is tried before "s"
SUFFIXES = sorted(["ations", "ation", "ingly", "edly", "ing",
                   "ied", "ies", "ers", "er", "ed", "ly", "s"], key=len,
reverse=True)

SAMPLE_TEXT = """Information retrieval is the science of searching for
```

```
information in documents.
A retrieval system indexes documents so that searching is fast. Indexing
builds an index
of terms. Users searched the index and retrieved documents. Retrieving
relevant documents
is the main goal, and good systems rank the retrieved documents. Search
engines index
millions of documents and return results for every search."""


def stem(word):
    """Very simple suffix-stripping stemmer (conflates word variants to one
stem)."""
    for suffix in SUFFIXES:
        if suffix == "s" and word.endswith("ss"):   # do not cut 'class' ->
'clas'
            continue
        if word.endswith(suffix) and len(word) - len(suffix) >= 3:
            word = word[:-len(suffix)]
            if suffix in ("ies", "ied"):            # studies -> study
                word += "y"
            break
    if word.endswith("e") and len(word) > 3:        # retrieve / retrieving -
> retriev
        word = word[:-1]
    return word


def main():
    path = Path("sample.txt")
    if not path.exists():
        path.write_text(SAMPLE_TEXT, encoding="utf-8")   # create demo file
on first run
    text = path.read_text(encoding="utf-8")

    words = re.findall(r"[a-z]+", text.lower())          # 1. tokenize
    total = len(words)
    words = [w for w in words if w not in STOP_WORDS]    # 2. remove stop
words

    counts = Counter()
    variants = defaultdict(set)
    for w in words:                                      # 3. stem + 4. count
```

```
        s = stem(w)
        counts[s] += 1
        variants[s].add(w)

    print(f"Total words            : {total}")
    print(f"After stop-word removal: {len(words)}")
    print(f"Unique stems           : {len(counts)}\n")

    print("Conflation classes (stem -> word variants):")
    for s, vs in sorted(variants.items()):
        if len(vs) > 1:
            print(f"  {s:10} -> {sorted(vs)}")

    TOP_N = 5
    print(f"\nDocument representative (top {TOP_N} terms):")
    for term, freq in counts.most_common(TOP_N):
        print(f"  {term:10} {freq}")


if __name__ == "__main__":
    main()
```

Output

```
Total words            : 60
After stop-word removal: 40
Unique stems           : 22

Conflation classes (stem -> word variants):
  index      -> ['index', 'indexes', 'indexing']
  retriev    -> ['retrieved', 'retrieving']
  search     -> ['search', 'searched', 'searching']
  system     -> ['system', 'systems']

Document representative (top 5 terms):
  document   6
  search     5
  index      5
  retriev    3
  inform     2
```

Note for viva: this is a simple suffix stripper, so it is not perfect ("retrieval" stays separate from "retrieve"). The Porter stemmer from NLTK is the standard, more accurate version.

Practical 2: Single-pass clustering of files

Aim: group 4 to 5 text files into clusters of similar content in one scan.

Idea: each document is compared with the existing clusters. If it is similar enough to one it joins it, otherwise it starts a new cluster. Similarity is measured with cosine similarity.

Steps

1. Put 4 or 5 `.txt` files in the `documents` folder.
2. Read each file, lowercase it, remove stop words, and count words to get a term-frequency vector.
3. Set a similarity threshold (0.25 here).
4. Take the first document and make it Cluster 1.
5. For every next document, compute cosine similarity with each cluster's centroid.
6. If the best similarity is at least the threshold, add the document to that cluster and add its word counts to the centroid.
7. Otherwise create a new cluster for it.
8. Print each cluster with its files and top terms.

Code (`p2_single_pass.py`)

```
import math
import re
from collections import Counter
from pathlib import Path

THRESHOLD = 0.25  # minimum cosine similarity to join an existing cluster
STOP_WORDS = {"a", "an", "the", "is", "are", "was", "were", "of", "and",
"or",
              "to", "in", "on", "for", "with", "as", "by", "at", "it",
"this",
              "that", "from", "be", "can", "has", "have", "its", "into"}


def tokenize(text):
    """Lowercase, keep only alphabetic words, drop stop words."""
    words = re.findall(r"\b[a-z]+\b", text.lower())
    return [w for w in words if w not in STOP_WORDS]


def cosine(v1, v2):
```

```
    """Cosine similarity between two term-frequency dictionaries."""
    common = set(v1) & set(v2)
    dot = sum(v1[w] * v2[w] for w in common)
    n1 = math.sqrt(sum(x * x for x in v1.values()))
    n2 = math.sqrt(sum(x * x for x in v2.values()))
    return dot / (n1 * n2) if n1 and n2 else 0.0


def single_pass(docs, threshold=THRESHOLD):
    """docs: list of (name, term-frequency Counter). Returns list of
clusters."""
    clusters = []  # each: {"members": [names], "centroid": Counter}
    for name, vec in docs:                      # ONE pass over the documents
        best, best_sim = None, 0.0
        for c in clusters:
            sim = cosine(vec, c["centroid"])
            if sim > best_sim:
                best, best_sim = c, sim
        if best is not None and best_sim >= threshold:
            best["members"].append(name)
            best["centroid"].update(vec)        # add terms -> update
centroid
        else:
            clusters.append({"members": [name], "centroid": Counter(vec)})
    return clusters


def main():
    folder = Path("documents")
    files = sorted(folder.glob("*.txt"))[:5]    # use 4-5 files
    if not files:
        print("No text files found in the 'documents' folder.")
        raise SystemExit

    docs = []
    for f in files:
        text = f.read_text(encoding="utf-8")
        docs.append((f.name, Counter(tokenize(text))))

    clusters = single_pass(docs)

    print(f"Threshold = {THRESHOLD}\n")
    for i, c in enumerate(clusters, 1):
```

```
        top = [w for w, _ in c["centroid"].most_common(5)]
        print(f"Cluster {i}: {c['members']}")
        print(f"   Top terms: {top}")


if __name__ == "__main__":
    main()
```

Output (using the five sample files from Setup)

```
Threshold = 0.25

Cluster 1: ['d1.txt', 'd2.txt', 'd5.txt']
   Top terms: ['match', 'cricket', 'batting', 'bowling', 'stadium']
Cluster 2: ['d3.txt', 'd4.txt']
   Top terms: ['python', 'machine', 'learning', 'data', 'programming']
```

d5 (football) joins the cricket cluster because it shares "match" and "stadium". A higher threshold such as 0.4 would separate it.

Note for viva: results depend on the order of the files, because each document is placed once and never moved.

Practical 3: Document retrieval using inverted files

Aim: build an inverted file (inverted index) for a set of documents and retrieve the documents that match a query.

Idea: instead of scanning every document for each query, an inverted file stores each word once, with the list of documents that contain it and how often. A query then only looks up its own words.

Steps

1. Read every `.txt` file in the `documents` folder.
2. Tokenize each file and remove stop words.
3. For each word, record the file name and the number of times it occurs, giving `word -> {file: count}`.
4. Read the user's query and tokenize it the same way.
5. Look up each query word in the index to get its set of documents.
6. For an AND query take the intersection of the sets. For an OR query take the union.

7. Rank the documents by the total occurrences of the query words and print them.

Code (`p3_inverted_index.py` , uses the same `documents` folder as Practical 2)

```
import re
from collections import defaultdict
from pathlib import Path

STOP_WORDS = {"a", "an", "the", "is", "are", "was", "were", "of", "and",
"or", "to",
              "in", "on", "for", "with", "as", "by", "at", "it", "this",
"that",
              "from", "be", "can", "has", "have", "its", "into"}


def tokenize(text):
    words = re.findall(r"[a-z]+", text.lower())
    return [w for w in words if w not in STOP_WORDS]


def build_index(folder):
    """Inverted file:  term -> {document name: how many times term occurs}"""
    index = defaultdict(dict)
    for f in sorted(Path(folder).glob("*.txt")):
        for word in tokenize(f.read_text(encoding="utf-8")):
            index[word][f.name] = index[word].get(f.name, 0) + 1
    return index


def search(index, query, mode="AND"):
    """AND = documents containing all terms, OR = documents containing any
term."""
    terms = tokenize(query)
    if not terms:
        return []
    doc_sets = [set(index.get(t, {})) for t in terms]
    docs = set.intersection(*doc_sets) if mode == "AND" else
set.union(*doc_sets)
    # rank: total number of query-term occurrences in the document
    scores = {d: sum(index[t].get(d, 0) for t in terms if t in index) for d
in docs}
    return sorted(scores.items(), key=lambda x: (-x[1], x[0]))
```

```
def main():
    index = build_index("documents")
    if not index:
        print("No .txt files found in the 'documents' folder.")
        return

    print("INVERTED INDEX")
    for term in sorted(index):
        print(f"  {term:12} -> {index[term]}")

    print("\nSAMPLE SEARCHES")
    for q, mode in [("cricket match", "AND"), ("python data", "AND"), ("match
goal", "OR")]:
        print(f"  {mode} '{q}': {search(index, q, mode)}")

    print("\nTry your own (type 'quit' to stop). Prefix with 'or:' for OR
search.")
    while True:
        try:
            q = input("Query> ").strip()
        except EOFError:
            break
        if q.lower() == "quit":
            break
        mode = "AND"
        if q.lower().startswith("or:"):
            mode, q = "OR", q[3:]
        results = search(index, q, mode)
        print("  Results:", results if results else "No document found")


if __name__ == "__main__":
    main()
```

Output (first part of the index, then the searches)

```
INVERTED INDEX
  batting      -> {'d1.txt': 1, 'd2.txt': 1}
  bowling      -> {'d1.txt': 1, 'd2.txt': 1}
  cricket      -> {'d1.txt': 1, 'd2.txt': 1}
  data         -> {'d3.txt': 1, 'd4.txt': 1}
  match        -> {'d1.txt': 1, 'd2.txt': 1, 'd5.txt': 1}
  python       -> {'d3.txt': 1, 'd4.txt': 1}
  stadium      -> {'d1.txt': 1, 'd5.txt': 1}
  ... (one line per word)

SAMPLE SEARCHES
  AND 'cricket match': [('d1.txt', 2), ('d2.txt', 2)]
  AND 'python data': [('d3.txt', 2), ('d4.txt', 2)]
  OR 'match goal': [('d5.txt', 2), ('d1.txt', 1), ('d2.txt', 1)]
```

Note for viva: the index is built once and reused, so search time depends on the number of query words, not on the number of documents.

Practical 4: Precision and recall

Aim: calculate precision and recall for a query q1, given the answer set A and the set of relevant documents Rq1.

Idea: precision asks how many retrieved documents are relevant. Recall asks how many relevant documents were retrieved. Let Ra be the relevant documents found inside A.

```
\text{Precision} = \frac{|R_a|}{|A|} \qquad \text{Recall} = \frac{|R_a|}
{|R_{q1}|}
```

Steps

1. Store the answer set A (documents the system returned) as a Python set.
2. Store the relevant set Rq1 (documents that truly match q1) as another set.
3. Find Ra, the documents in both sets, using the intersection `A & Rq1`.
4. Compute precision as |Ra| divided by |A|.
5. Compute recall as |Ra| divided by |Rq1|.
6. Print both values.

Code (`p4_precision_recall.py`)

```
# A   = answer set (documents retrieved by the system for query q1)
# Rq1 = documents that are truly relevant to q1

A = {"d123", "d84", "d56", "d6", "d8", "d9", "d511", "d129",
     "d187", "d25", "d38", "d48", "d250", "d113", "d3"}
Rq1 = {"d3", "d5", "d9", "d25", "d39", "d44", "d56", "d71", "d89", "d123"}

Ra = A & Rq1                       # relevant documents that were actually
retrieved

precision = len(Ra) / len(A)       # retrieved documents that are relevant
recall = len(Ra) / len(Rq1)        # relevant documents that were retrieved

print("Answer set A          :", sorted(A), f"(|A| = {len(A)})")
print("Relevant set Rq1      :", sorted(Rq1), f"(|R| = {len(Rq1)})")
print("Relevant & retrieved  :", sorted(Ra), f"(|Ra| = {len(Ra)})")
print(f"Precision = |Ra|/|A|  = {len(Ra)}/{len(A)} = {precision:.4f}
({precision:.0%})")
print(f"Recall    = |Ra|/|R|  = {len(Ra)}/{len(Rq1)} = {recall:.4f}
({recall:.0%})")
```

Output

```
Answer set A          : ['d113', 'd123', 'd129', 'd187', 'd25', 'd250', 'd3',
'd38', 'd48', 'd511', 'd56', 'd6', 'd8', 'd84', 'd9'] (|A| = 15)
Relevant set Rq1      : ['d123', 'd25', 'd3', 'd39', 'd44', 'd5', 'd56',
'd71', 'd89', 'd9'] (|R| = 10)
Relevant & retrieved  : ['d123', 'd25', 'd3', 'd56', 'd9'] (|Ra| = 5)
Precision = |Ra|/|A|  = 5/15 = 0.3333 (33%)
Recall    = |Ra|/|R|  = 5/10 = 0.5000 (50%)
```

Reading the result: only 5 of the 15 retrieved documents are relevant (33% precision), and the system found half of the 10 relevant documents (50% recall). To use your own data, replace the two sets at the top.

Practical 5: Harmonic mean (F-measure) and E-measure

Aim: calculate the F-measure and E-measure for the same example as Practical 4.

Idea: the F-measure is the harmonic mean of precision (P) and recall (R), so it is high only when both are high. The E-measure is 1 minus F, so a smaller E is better. The value beta decides which one matters more: below 1 favours precision, above 1 favours recall.

```
F = \frac{2}{\frac{1}{R} + \frac{1}{P}} = \frac{2PR}{P+R} \qquad E = 1 - F
```

```
F_\beta = \frac{(1+\beta^2)\,P\,R}{\beta^2 P + R} \qquad E_\beta = 1 -
F_\beta
```

Steps

1. Reuse the sets A and Rq1 from Practical 4 and compute P and R.
2. Compute the harmonic mean F = 2 / (1/R + 1/P).
3. Compute E = 1 - F.
4. Define two small functions for the general beta versions.
5. Print F and E for beta = 0.5, 1 and 2 to show the effect of beta.

Code (`p5_f_and_e_measure.py`)

```
A = {"d123", "d84", "d56", "d6", "d8", "d9", "d511", "d129",
     "d187", "d25", "d38", "d48", "d250", "d113", "d3"}
Rq1 = {"d3", "d5", "d9", "d25", "d39", "d44", "d56", "d71", "d89", "d123"}

Ra = A & Rq1
P = len(Ra) / len(A)      # precision
R = len(Ra) / len(Rq1)    # recall


def f_measure(p, r, beta=1.0):
    """Harmonic mean of P and R. beta = 1 gives F1 = 2PR/(P+R)."""
    if p == 0 and r == 0:
        return 0.0
    return (1 + beta ** 2) * p * r / (beta ** 2 * p + r)


def e_measure(p, r, beta=1.0):
    """van Rijsbergen's E = 1 - F. Smaller E is better."""
    return 1 - f_measure(p, r, beta)


print(f"Precision P = {P:.4f}")
print(f"Recall    R = {R:.4f}\n")

# F-measure as the harmonic mean:  F = 2 / (1/R + 1/P)
F = 2 / (1 / R + 1 / P)
print(f"Harmonic mean  F = 2 / (1/R + 1/P) = {F:.4f}")
```

```
print(f"E-measure      E = 1 - F           = {1 - F:.4f}\n")

print("Effect of beta  (beta < 1 favours precision, beta > 1 favours
recall)")
print(f"{'beta':>6} {'F':>8} {'E':>8}")
for b in (0.5, 1, 2):
    print(f"{b:>6} {f_measure(P, R, b):>8.4f} {e_measure(P, R, b):>8.4f}")
```

Output

```
Precision P = 0.3333
Recall    R = 0.5000

Harmonic mean  F = 2 / (1/R + 1/P) = 0.4000
E-measure      E = 1 - F           = 0.6000

Effect of beta  (beta < 1 favours precision, beta > 1 favours recall)
  beta        F        E
   0.5   0.3571   0.6429
     1   0.4000   0.6000
     2   0.4545   0.5455
```

Reading the result: F = 0.40 and E = 0.60. F sits between precision (0.33) and recall (0.50) but closer to the lower one, which is how a harmonic mean behaves.

Practical 6: Feature extraction from a 2D colour image

Aim: extract colour and texture features from an image and plot their histograms.

Idea: an RGB image is a 3D array of numbers (height × width × 3 colour channels). Colour features describe how much of each colour there is. Texture features describe how pixel intensities vary. A histogram counts how many pixels have each intensity from 0 to 255.

Feature	Type	Meaning
Mean (R, G, B)	Colour	Average intensity of each channel
Standard deviation (R, G, B)	Colour	Spread of each channel
RGB histogram	Colour	Pixel count per intensity for each channel
Contrast	Texture	Standard deviation of the grayscale image

Feature	Type	Meaning
Entropy	Texture	Randomness of grayscale intensities
Edge strength	Texture	Average change between neighbouring pixels

Steps

1. Install the libraries: `pip install pillow numpy matplotlib`
2. Put an image in the folder and set `IMAGE_PATH` to its name. If the file is missing, the program makes a demo image.
3. Open the image with Pillow, convert it to RGB, and turn it into a NumPy array.
4. Compute the mean and standard deviation of each colour channel.
5. Compute a 256-bin histogram for the R, G and B channels.
6. Convert the image to grayscale, then compute contrast, entropy and edge strength.
7. Plot the image, the RGB histogram and the grayscale histogram side by side, and save the figure as `histogram.png`.

Code (`p6_image_features.py`)

```
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
from PIL import Image

IMAGE_PATH = "sample.jpg"          # change this to your own image


def make_sample_image(path):
    """Creates a demo image if none exists, so the program always runs."""
    rng = np.random.default_rng(1)
    h, w = 200, 300
    img = np.zeros((h, w, 3), dtype=np.uint8)
    img[:, :, 0] = np.linspace(0, 255, w)                # red rises left ->
right
    img[:, :, 1] = np.linspace(255, 0, h)[:, None]       # green falls top -
> bottom
    img[:, :, 2] = rng.integers(0, 120, (h, w))          # random blue =
texture
    Image.fromarray(img).save(path)
```

```
if not Path(IMAGE_PATH).exists():
    make_sample_image(IMAGE_PATH)

img = Image.open(IMAGE_PATH).convert("RGB")
rgb = np.array(img)                                          # shape (height,
width, 3)
gray = np.array(img.convert("L"))                            # grayscale version

# ---------- COLOUR FEATURES ----------
pixels = rgb.reshape(-1, 3)
mean = pixels.mean(axis=0)
std = pixels.std(axis=0)
hist_rgb = [np.histogram(rgb[:, :, i], bins=256, range=(0, 256))[0] for i in
range(3)]

# ---------- TEXTURE FEATURES ----------
hist_gray = np.histogram(gray, bins=256, range=(0, 256))[0]
p = hist_gray / hist_gray.sum()
entropy = -np.sum(p[p > 0] * np.log2(p[p > 0]))           # randomness of
intensities
contrast = gray.std()                                         # spread of
intensities
g = gray.astype(float)
edge_strength = (np.abs(np.diff(g, axis=1)).mean() + np.abs(np.diff(g,
axis=0)).mean()) / 2

print(f"Image size : {img.size[0]} x {img.size[1]} pixels")
print(f"Mean  (R,G,B) : {mean.round(2)}")
print(f"Std   (R,G,B) : {std.round(2)}")
print(f"Contrast      : {contrast:.2f}")
print(f"Entropy       : {entropy:.2f}")
print(f"Edge strength : {edge_strength:.2f}")

# ---------- PLOT ----------
fig, ax = plt.subplots(1, 3, figsize=(15, 4))
ax[0].imshow(rgb)
ax[0].set_title("Input image")
ax[0].axis("off")

for h, colour, name in zip(hist_rgb, ("red", "green", "blue"), ("R", "G",
"B")):
```

```
    ax[1].plot(h, color=colour, label=name)
ax[1].set_title("RGB colour histogram")
ax[1].set_xlabel("Intensity (0-255)")
ax[1].set_ylabel("Pixel count")
ax[1].legend()

ax[2].bar(range(256), hist_gray, color="gray", width=1.0)
ax[2].set_title("Grayscale histogram (texture)")
ax[2].set_xlabel("Intensity (0-255)")
ax[2].set_ylabel("Pixel count")

plt.tight_layout()
plt.savefig("histogram.png", dpi=120)
plt.show()
```

Output (on the demo image; your values will differ for your own image)

```
Image size : 300 x 200 pixels
Mean  (R,G,B) : [126.93 127.07  59.24]
Std   (R,G,B) : [73.94 73.96  7.96]
Contrast      : 48.78
Entropy       : 7.60
Edge strength : 1.98
```

The program also shows the three plots (image, RGB histogram, grayscale histogram) and saves them as `histogram.png`. On the demo image the blue histogram is a narrow peak near 60 because blue was generated only between 0 and 120 at random, while red and green are spread evenly across 0 to 255.

Note for viva: colour histograms are used in content-based image retrieval, where two images with similar histograms are treated as similar.

Practical 7: Web crawler for product information and links

Aim: crawl an e-commerce website and collect each product's title, price, availability and link.

Idea: a crawler downloads a page, extracts the data it needs from the HTML, finds the link to the next page, and repeats. This program uses books.toscrape.com, a practice shop built for learning web scraping. Check a real shop's terms and robots.txt before crawling it.

Steps

1. Install the libraries: `pip install requests beautifulsoup4`
2. Open the site in a browser, right-click a product and choose Inspect. Each product sits inside `article.product_pod`, with the title and link in `h3 a`, the price in `p.price_color`, and the stock in `p.availability`.
3. Download a listing page with `requests.get()` and set `response.encoding = "utf-8"` so the £ sign is read correctly.
4. Parse the HTML with BeautifulSoup and extract title, price, availability and link for each product.
5. Convert each relative link to a full URL with `urljoin`.
6. Find the "next" button (`li.next a`) and repeat from step 3 until `MAX_PAGES` is reached.
7. Wait one second between requests so the server is not overloaded.
8. Save all products to `products.csv` and print the first few.

Code (`p7_web_crawler.py`, needs internet access)

```
import csv
import time
from urllib.parse import urljoin

import requests
from bs4 import BeautifulSoup

START_URL = "https://books.toscrape.com/catalogue/page-1.html"
MAX_PAGES = 3                                    # how many listing pages to
crawl
HEADERS = {"User-Agent": "Mozilla/5.0 (student practical crawler)"}


def parse_products(html, page_url):
    """Return (list of products on this page, URL of the next page or
None)."""
    soup = BeautifulSoup(html, "html.parser")
    products = []
    for item in soup.select("article.product_pod"):
        a = item.h3.a
        products.append({
            "title": a["title"],
```

```
            "price": item.select_one("p.price_color").get_text(strip=True),
            "availability":
item.select_one("p.availability").get_text(strip=True),
            "link": urljoin(page_url, a["href"]),          # make link
absolute
        })
    nxt = soup.select_one("li.next a")
    next_url = urljoin(page_url, nxt["href"]) if nxt else None
    return products, next_url


def crawl(start_url, max_pages):
    all_products, url, page = [], start_url, 1
    while url and page <= max_pages:
        print(f"Crawling page {page}: {url}")
        response = requests.get(url, headers=HEADERS, timeout=15)
        response.raise_for_status()
        response.encoding = "utf-8"                        # keeps the £ sign
correct
        products, url = parse_products(response.text, url)
        all_products.extend(products)
        page += 1
        time.sleep(1)                                      # be polite to the
server
    return all_products


def save_csv(products, filename="products.csv"):
    with open(filename, "w", newline="", encoding="utf-8-sig") as f:
        writer = csv.DictWriter(f, fieldnames=["title", "price",
"availability", "link"])
        writer.writeheader()
        writer.writerows(products)


def main():
    products = crawl(START_URL, MAX_PAGES)
    save_csv(products)
    print(f"\nCollected {len(products)} products. Saved to products.csv\n")
    for p in products[:5]:
        print(f"{p['title']} | {p['price']} | {p['availability']}")
        print(f"   {p['link']}")
```

```
if __name__ == "__main__":
    main()
```

Expected output (3 pages of 20 products each; the first product on this site is "A Light in the Attic")

```
Crawling page 1: https://books.toscrape.com/catalogue/page-1.html
Crawling page 2: https://books.toscrape.com/catalogue/page-2.html
Crawling page 3: https://books.toscrape.com/catalogue/page-3.html

Collected 60 products. Saved to products.csv

A Light in the Attic | £51.77 | In stock
   https://books.toscrape.com/catalogue/a-light-in-the-attic_1000/index.html
...
```

Testing note: my workspace has no internet access, so I could not run this against the live site. I tested the parsing function on a sample page with the same HTML structure, and it returned the correct titles, prices, full links and next-page URL. Run it once on your own computer, and if the site layout has changed, only the selectors inside `parse_products` need adjusting.

Note for viva: a crawler can also follow each product link to read its detail page. Real shops often block crawlers or need JavaScript rendering, in which case tools such as Selenium are used.

Practical 8: Live weather report of a city

Aim: take a city name and display its current temperature, wind speed, weather and description.

Idea: a weather API returns current data in JSON when you send it a location. This program uses Open-Meteo, which is free and needs no API key. It works in two calls: first the city name is converted to latitude and longitude, then those are used to fetch the current weather.

Steps

1. Install the library: `pip install requests`
2. Ask the user for a city name.

3. Call the geocoding API with the city name to get its latitude and longitude. If nothing comes back, print "City not found".
4. Call the forecast API with the latitude and longitude, asking for the current temperature, humidity, wind speed and weather code.
5. Convert the numeric weather code into a weather group (such as Clear or Rain) and a description using the `WEATHER_CODES` table.
6. Print the report. Wrap the calls in `try/except` so a network error gives a clear message.

Code (`p8_weather.py` , needs internet access)

```
import requests

GEO_URL = "https://geocoding-api.open-meteo.com/v1/search"
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

# weather_code -> (weather, description)
WEATHER_CODES = {
    0: ("Clear", "Clear sky"), 1: ("Clear", "Mainly clear"),
    2: ("Clouds", "Partly cloudy"), 3: ("Clouds", "Overcast"),
    45: ("Fog", "Fog"), 48: ("Fog", "Depositing rime fog"),
    51: ("Drizzle", "Light drizzle"), 53: ("Drizzle", "Moderate drizzle"),
    55: ("Drizzle", "Dense drizzle"),
    61: ("Rain", "Slight rain"), 63: ("Rain", "Moderate rain"), 65: ("Rain",
"Heavy rain"),
    71: ("Snow", "Slight snow"), 73: ("Snow", "Moderate snow"), 75: ("Snow",
"Heavy snow"),
    80: ("Rain", "Slight rain showers"), 81: ("Rain", "Moderate rain
showers"),
    82: ("Rain", "Violent rain showers"),
    95: ("Thunderstorm", "Thunderstorm"), 96: ("Thunderstorm", "Thunderstorm
with hail"),
    99: ("Thunderstorm", "Thunderstorm with heavy hail"),
}


def get_weather(city):
    # Step 1: city name -> latitude and longitude
    geo = requests.get(GEO_URL, params={"name": city, "count": 1},
timeout=10)
    geo.raise_for_status()
    results = geo.json().get("results")
```

```
    if not results:
        return None
    place = results[0]

    # Step 2: latitude/longitude -> current weather
    resp = requests.get(WEATHER_URL, timeout=10, params={
        "latitude": place["latitude"],
        "longitude": place["longitude"],
        "current":
"temperature_2m,relative_humidity_2m,wind_speed_10m,weather_code",
        "timezone": "auto",
    })
    resp.raise_for_status()
    now = resp.json()["current"]
    weather, description = WEATHER_CODES.get(now["weather_code"], ("Unknown",
"Unknown"))
    return {
        "city": f"{place['name']}, {place.get('country', '')}",
        "temperature": now["temperature_2m"],
        "wind_speed": now["wind_speed_10m"],
        "humidity": now["relative_humidity_2m"],
        "weather": weather,
        "description": description,
    }


def main():
    city = input("Enter city name: ").strip()
    try:
        data = get_weather(city)
    except requests.RequestException as e:
        print("Network/API error:", e)
        return
    if data is None:
        print("City not found.")
        return
    print(f"\nWeather report for {data['city']}")
    print(f"  Temperature : {data['temperature']} °C")
    print(f"  Wind speed  : {data['wind_speed']} km/h")
    print(f"  Humidity    : {data['humidity']} %")
    print(f"  Weather     : {data['weather']}")
    print(f"  Description : {data['description']}")
```

```
if __name__ == "__main__":
    main()
```

Example output (the numbers change with the live weather; these are sample values)

```
Enter city name: Pune

Weather report for Pune, India
  Temperature : 27.4 °C
  Wind speed  : 9.3 km/h
  Humidity    : 61 %
  Weather     : Clouds
  Description : Partly cloudy
```

Testing note: my workspace has no internet access, so I could not call the live API. I ran the program with the two API responses replaced by sample data in the same format, and it produced the output above, including the "City not found" case.

Alternative version with OpenWeatherMap. If your teacher expects this service, create a free key at openweathermap.org (new keys can take some time to activate) and use this shorter program. Its response already contains the weather group and description.

```
import requests

API_KEY = "PASTE_YOUR_KEY_HERE"
city = input("Enter city name: ").strip()

url = "https://api.openweathermap.org/data/2.5/weather"
r = requests.get(url, params={"q": city, "appid": API_KEY, "units":
"metric"}, timeout=10)
data = r.json()

if r.status_code != 200:
    print("Error:", data.get("message", "could not fetch weather"))
else:
    print(f"Weather report for {data['name']}")
    print(f"  Temperature : {data['main']['temp']} °C")
    print(f"  Wind speed  : {data['wind']['speed']} m/s")
    print(f"  Weather     : {data['weather'][0]['main']}")
    print(f"  Description : {data['weather'][0]['description']}")
```

I did not run this version, since it needs your key and internet access.

Practical 9: Case study on a recommender system (music)

Aim: study how a recommender system suggests items to a user, using music as the example, and implement a small working version.

Problem: a music app has millions of songs and every listener has different taste. A recommender system predicts which songs a user has not heard yet but is likely to enjoy, using the user's past behaviour and the behaviour of similar users.

Types of recommender systems

Type	How it works	Strength	Weakness
Content-based	Recommends songs with features (genre, artist, tempo) similar to what the user liked	Needs no other users; works for new songs	Suggests only more of the same; needs good song features
Collaborative filtering	Recommends songs liked by users with similar ratings	Finds surprising songs; needs no song features	Cold start: fails for new users and new songs
Hybrid	Combines both approaches	Fewer weaknesses of either	More complex to build

The program below uses user-based collaborative filtering, the easiest to implement and explain.

Data: a ratings matrix with 5 users and 6 songs. Ratings go from 1 to 5, and 0 means the user has not rated that song.

Steps

1. Build the user-song ratings matrix. The target user (Rohit) has rated only Song A and Song B.
2. Compute the cosine similarity between the target user and every other user.
3. For each song the target user has not rated, take the ratings of the users who did rate it.
4. Predict the rating as a similarity-weighted average: users more like the target count more.

5. Sort the songs by predicted rating and recommend the top N.

Code (`p9_music_recommender.py`)

```
import numpy as np

songs = ["Song A", "Song B", "Song C", "Song D", "Song E", "Song F"]
users = ["Asha", "Ben", "Chen", "Divya", "Rohit"]

# rows = users, columns = songs, 0 means "not rated yet"
ratings = np.array([
    [5, 4, 0, 0, 1, 0],    # Asha
    [4, 5, 1, 0, 0, 2],    # Ben
    [1, 0, 5, 4, 0, 5],    # Chen
    [0, 1, 4, 5, 2, 4],    # Divya
    [5, 5, 0, 0, 0, 0],    # Rohit (target user)
], dtype=float)


def cosine(a, b):
    denom = np.linalg.norm(a) * np.linalg.norm(b)
    return float(a @ b / denom) if denom else 0.0


def recommend(user_name, top_n=3):
    u = users.index(user_name)
    # 1. similarity of the target user with every other user
    sims = {v: cosine(ratings[u], ratings[v]) for v in range(len(users)) if v
!= u}

    # 2. predict a score for every song the target user has not rated
    predictions = {}
    for j, song in enumerate(songs):
        if ratings[u, j] != 0:
            continue
        num = sum(s * ratings[v, j] for v, s in sims.items() if ratings[v, j]
!= 0)
        den = sum(s for v, s in sims.items() if ratings[v, j] != 0)
        if den:
            predictions[song] = num / den

    # 3. highest predicted rating first
    return sims, sorted(predictions.items(), key=lambda x: -x[1])[:top_n]
```

```
if __name__ == "__main__":
    target = "Rohit"
    sims, recs = recommend(target)
    print(f"Similarity of {target} with other users:")
    for v, s in sorted(sims.items(), key=lambda x: -x[1]):
        print(f"  {users[v]:6} {s:.3f}")
    print(f"\nTop recommendations for {target}:")
    for song, score in recs:
        print(f"  {song}  (predicted rating {score:.2f})")
```

Output

```
Similarity of Rohit with other users:
  Asha   0.982
  Ben    0.938
  Divya  0.090
  Chen   0.086

Top recommendations for Rohit:
  Song D  (predicted rating 4.51)
  Song F  (predicted rating 2.39)
  Song C  (predicted rating 1.55)
```

Result: Rohit is very similar to Asha and Ben, who rated Songs A and B highly, and unlike Chen and Divya. Song D scores highest because Chen and Divya rated it highly, and the program still recommends it as the top result. This is a weakness of the simple weighted average: when only a few users rated a song, their opinion decides the score even if they are dissimilar. A better version uses only the most similar users (the k nearest neighbours), which is a good improvement to mention.

Evaluation: recommender systems are measured with RMSE or MAE for predicted ratings, and with precision and recall (Practicals 4 and 5) for the top-N list.

Limitations and challenges

- Cold start: a new user or a new song has no ratings.
- Sparsity: most users rate only a tiny fraction of all songs.
- Popularity bias: popular songs get recommended more and more.
- Privacy: listening history is personal data.

Real-world use: music apps, video streaming services and online shops all use recommenders, usually hybrids of content-based and collaborative methods. The same method also works for products, doctors or prices by changing what the columns of the matrix are.